AI Fitness Plan Coach
GUI: User profile page, workout plan calendar, workout tracker, progress dashboard, coach chatbox, nutrition page, weekly report page, workout history import page
AI: AI builds a personalized workout plan based on user goal and information. Searches user workout data to adapt plans to changes in weight/reps and exercises. The exercise plans are thoroughly checked by the AI to meet safety guidelines. The AI can answer questions that users may have about their exercise goals but proper medical advice must be given by a doctor as the advice is only general.
Features: user profile boundaries; workout plan generation; user-defined changes to the workout plan; exercise swapping; workout tracker; automatic weight progression; daily check-in; progress dashboard; coach chatbox; nutrition and meals; weekly progress report; import workout history.

1.1 Project Description
Clearly describe:
• What problem does your project solve?

I have decided for my project to make an AI fitness help bot that promotes physical exercise/conditioning to the users.

There are two major problems that my project will solve.

- Firstly, most current workout apps are not properly tailored to the consumer. Most current apps do not have flexibility when it comes to breaks/plans one would make calling fitness a 'commitment' that one must uphold no matter what instead of promoting the idea that fitness starts with small habits and can be a gradual journey. To further promote the idea that fitness can have a different definition for each person (For example a body builder's workout schedule is very different than that of a beginner that just started and is not used to strenuous exercise.) the AI takes in mind your availability and what days suit you. It also takes into account the equipment one would have and also any past injuries/disabilities that may affect current exercise range. The app will also be able to flexibly change your workout plan if any of these conditions change.
- The other problem I aim to address is the fact that even though most apps gain so much data from workouts and user input, behind the scenes most apps rarely use such data for further calculations and rely on generalisations instead. Therefore my app will allow for data to be used to its fullest and give people a place where they can get reliable information tailored to their current lifestyle.

• Who are the target users?

There are 3 main target users for my project:

- First would be those who are more inexperienced gym-goers. This can include those who are beginners or those who have gone for a while but still do not have a structured gym schedule in place.
- Then there would be those that workout at home as they are usually limited in the equipment they can choose to use therefore the app will be able to suggest to them good workouts either with the equipment they have or bodyweight exercises.

- There would also be those with extremely flexible schedules such as students or variable hour occupations. The app will be able to match the times that they are available or fit their changing schedule/equipment (equipment can change in an occupation that has a lot of traveling such as flight attendants).

• What can the agent do?

The agent is capable of building a training program for you based on what goal you are working to. Within this goal, it is capable of taking into consideration any constraints you may have such as time, your equipment, any injuries you may have, and also your recovery rate. It can decide when it believes that you can start to increase the weight in your workouts for progressive overload or if you need to use the same or even decrease weight if it is too hard. If any of your conditions may change, the project will be able to find what part of the program is affected by this change and accordingly revise the plan.

• Why is an AI agent appropriate for this problem?

An AI agent is appropriate for this project due to the nature of the problems I am solving. As the app is supposed to build a customizable plan around many different user conditions, it is almost impossible to predict all sorts of issues that may come up in a user's life such as different injuries and related recovery needed for them or changing workout schedule time & equipment while still trying to hit the equivalent progressive overload as required for growth. I believe that this project is supposed to show the difference between general fitness advice and advice that is tailored to you. As the AI agent must also use the past history of workouts by the user as well while also taking into account the users current requests, it is difficult to produce similar results without an AI agent and would need hundreds of different plug-ins to solve.

• Which AI/LLM model(s) do you plan to use?

I plan to use 2 AI models.

- The primary AI I will be using is Claude Sonnet 5 as I already have a Claude plan and some experience with integrating Claude into projects.
- The other AI I will be using is Llama for offline work in case the app cannot connect to Claude. While looking into app development for such AI agents I realized that it is important to have a local fallback model to make sure that people can still use your services in case of an issue with Claude or if you are not able to connect to the internet with llama being one of the best local lightweight models.

• How will the AI model interact with the rest of the software system?

The AI will interact with the rest of the system by being the main focus when a user requests a workout program. The AI will remember anything needed for the workout program to fit the user as discussed earlier such as any injuries or schedules that it must abide by and make the user a proper committable plan. When needed, the AI will look through other stored data such as past exercises given to the user, whether there is a calorie goal the user is trying to reach, whether there was an exercise that they struggled with, or any other relevant data that may be stored about the users preferences and past performances. Finally, the AI will

check if the data that it generated is safe for injured areas to workout and any other issues if there was any hallucination caused by the AI before fixing it and displaying to the user its final workout plan.

1.2 Feature Specification
Identify and describe at least 10 major features.
For each feature, specify:
1. Feature ID and Name
2. Description — What does the feature do?
3. User Interaction — How does the user access or use this feature through the GUI?
4. Input — What information does the feature require?
5. Output — What does it produce?
6. AI Involvement — Is the feature deterministic, AI-based, or hybrid?
7. Expected Workflow — What happens when the feature is executed?
8. Error/Alternative Cases — What could go wrong and how should the system respond?

F01 - User Profile Boundaries
On the user's profile account in the GUI, the user can enter basic information about themselves such as weight, age, height and workout experience. Other than basic workout information, they can also add anything that they believe that the AI planner should know such as any previous injuries, their workout goal, their equipment, and what dates/times they are available to workout. The AI reads this text and asks the user relevant questions such as extent of injury and specifics in how long each workout length should be and saves the finished profile to use for any future workout plan generation and for user assurance that the AI did not misunderstand user input.

F02 - Workout Plan Generation
The user can enter how many weeks they are willing to invest into a workout plan and clicks on the Workout Plan GUI to generate a plan. The AI looks at the user profile boundary, searches the exercise library for exercises that match their current equipment and information and builds a training plan for the user. The AI double checks the plan and asks the user to confirm that the plan works for them before saving it to the user's account under the calendar GUI. If the plan is not accepted by the user or there is found to be a boundary that was not taken in place when generating the plan, it will be regenerated.

F03 - User Defined Changes To The Workout Plan
The user can decide whether there is a need to change their workout plan on a whim in the Workout Plan GUI if there is a quick adjustment needed to their plan. For example, the user can ask to only train for 30 minutes instead of an hour on Tuesday or can say that next week their equipment will be limited as their bench broke and a new one is shipping now. The AI reads their message, finds the issues with their current workout plan and at what time period on the calendar and adjusts it accordingly. The GUI then asks users to confirm/undo changes to the schedule or to regenerate the request/change the parameters.

F04 - Exercise Swapping
If the user experiences pain, does not enjoy an exercise, or the machine is just being used, they can choose to swap the current exercise with a new one under the Workout Plan GUI.

The user can provide a reason for this change and the app will then find different exercise options based on this reason and will prompt the user to choose one of the three options for a new exercise with justification for all three of the choices. Then the user will choose whether this change is just for today or for the rest of the workout plan.

F05 - Workout Tracker
The user can press to start their workout on the Workout GUI and the app will go through each of today's exercises one at a time. For each exercise, the user enters the amount of reps they were able to reach, the current weight, and the difficulty from a scale of 1-10 with a rest timer between each set. When the workout is completed, the app will give a quick summary of the weight lifted and any records the user may have broken. Then the data will be saved so that the AI can use it to plan future workouts.

F06 - Automatic Weight Progression
After each workout, the app decides what the user's lift weight should be next time. For normal cases in which the user keeps hitting a high rep range, it will recommend to increase the weight. But for cases such as the user being stuck at the same weight for a few weeks or suddenly becoming weaker, the AI will analyze the user's recent workouts and inputs. It will then decide whether the user should enter a "deloading" period where weight is decreased for a week or two to let the muscles rest or to change the exercise altogether. These changes can be overridden by the user on the GUI if the user does not like them.

F07 - Daily CheckIn
Before starting each workout, the user answers some questions about their current physical condition such as sleep, energy, stress, etc. They can also add their own comments about their current physical condition and this allows for the app to translate these answers into a score to judge their current physical condition and choose whether to decrease or increase today's workout intensity and explain its reasoning. This can also be skipped by the user or rejected by the user to go on with a normal workout. If the current physical condition answers sounds grave, the app will recommend for the user to visit a doctor.

F08 - Progress Dashboard
This GUI feature will show the user's progress with charts showing how their strength & physicality has changed, what muscle groups they train the most/least, and other body measurements. The charts update automatically after every workout and are always accurate as they are calculated straight from the user's saved data/exercise data.

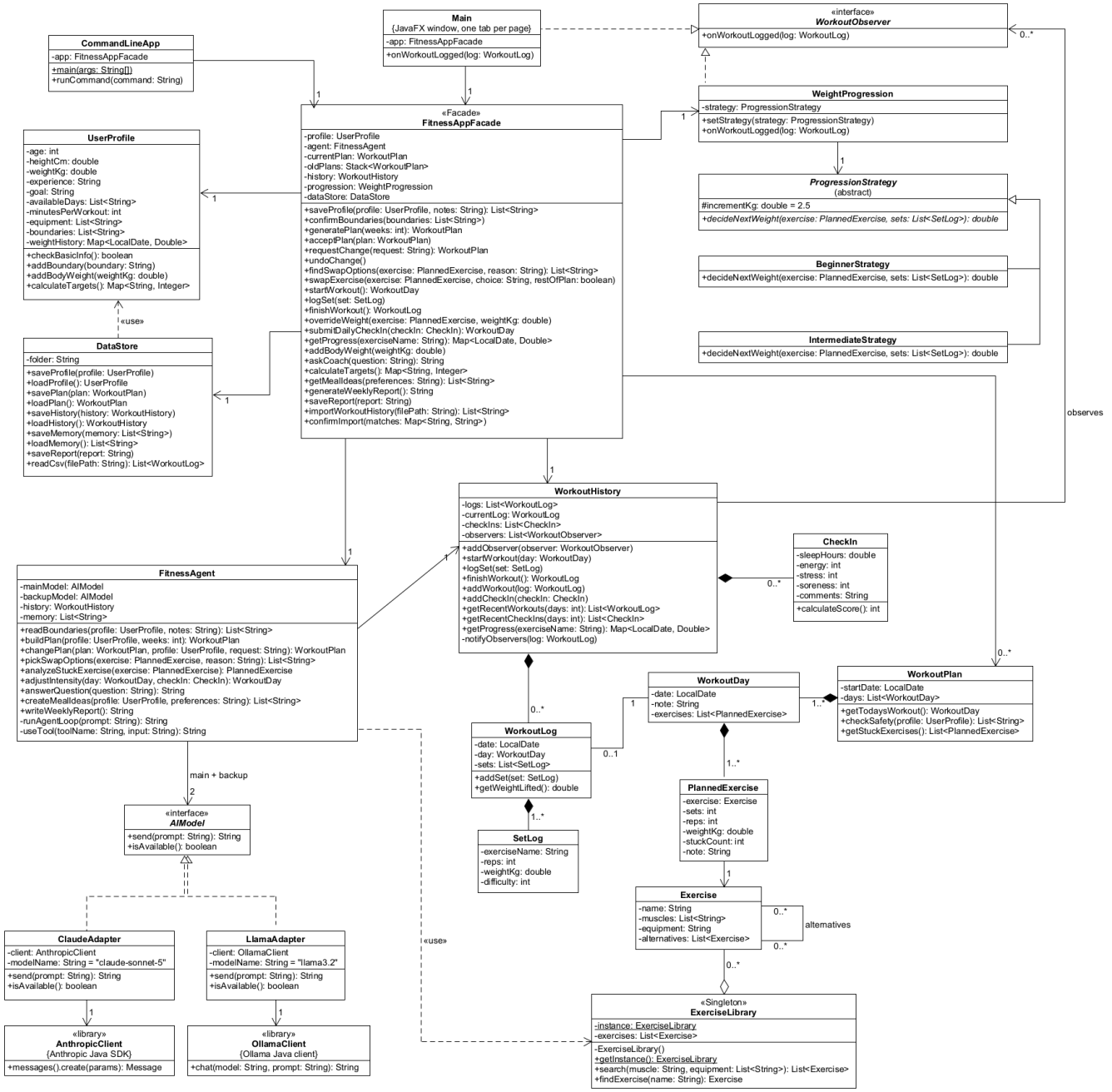
F09 - Coach Chatbox
This coach chatbox is used to answer any questions the user may have about their workouts using the GUI to communicate. This can range from asking about their progress in an exercise to an explanation on their current plan and how it will help them reach their goals. The AI searches the user's workout history, the exercise database, and user information in order to give general fitness advice to users (Only general fitness advice and all medical advice is recommended for a doctor). The answers show direct proof to their claims by showing where the information is coming from on their chart and can keep track of their conversations for next time.

F10 - Nutrition and Meals
On the nutrition GUI, users can calculate their recommended calories and macronutrients that the user should be aiming for. The calculation is based on the user information such as their body measurements and weight, activity level, and their current goal. The user can enter any food preferences or dislikes/allergies and can ask for meal ideas where the AI will prepare ideas that the user can make. The AI will double check for any missed food restrictions before handing the request to the user. The user can choose to accept the idea or regenerate it if they do not enjoy the food.

F11 - Weekly Progress Report
At the end of each week, the user is generated a report on the Report GUI which creates a short review to see what progress they made with their workouts, how their week of daily check-ins matched, and will provide suggestions on how to improve their plan for the next week if needed. The user can choose to apply or deny these suggestions for their plan and can choose to save the report if they want.

F12 - Import Workout History
The user can choose to import workout history from a previous fitness app which will allow for the app to start off with a base of data on the user's past workouts. The app will read the CSV file that is imported from another app and will match it to the relevant exercises under its own library. Any of the exercises that cannot be found will be listed to the user and can be found manually.

2.1 Class Diagram



For every pattern used, explain:

- **What design problem does the pattern address?**
- **Which classes participate in the pattern?**
- **What role does each class play?**
- **Why is this pattern appropriate?**
- **What would be more difficult if the pattern were not used?**

Facade:

Facade deals with acting as the center of a ‘mess of code’. It acts as a barrier between the client requests through the command line and main GUI and reroutes it to the proper methods. In this case, FitnessAppFacade is the main facade method connecting together the Main and CommandLineApp as the clients of the requests and rerouting them to

FitnessAgent, WorkoutHistory, WeightProgression, WorkoutPlan, UserProfile, and DataStore which are the main ‘working’ classes that process information given by the clients through the facade. This pattern is appropriate for this case due to the complexity and sheer amount of tasks needed to be deployed by the user and the fact that there are two different client methods (Main and CommandLineApp). It is what lets both clients reach the same class calls and allows for a more organized system. Without it the code would be way more difficult as everything would have to be routed directly to both Main and CommandLineApp. It will also be more difficult to add new systems in place if a change is needed later as rather than just adding to the facade you will have to modify both the clients.

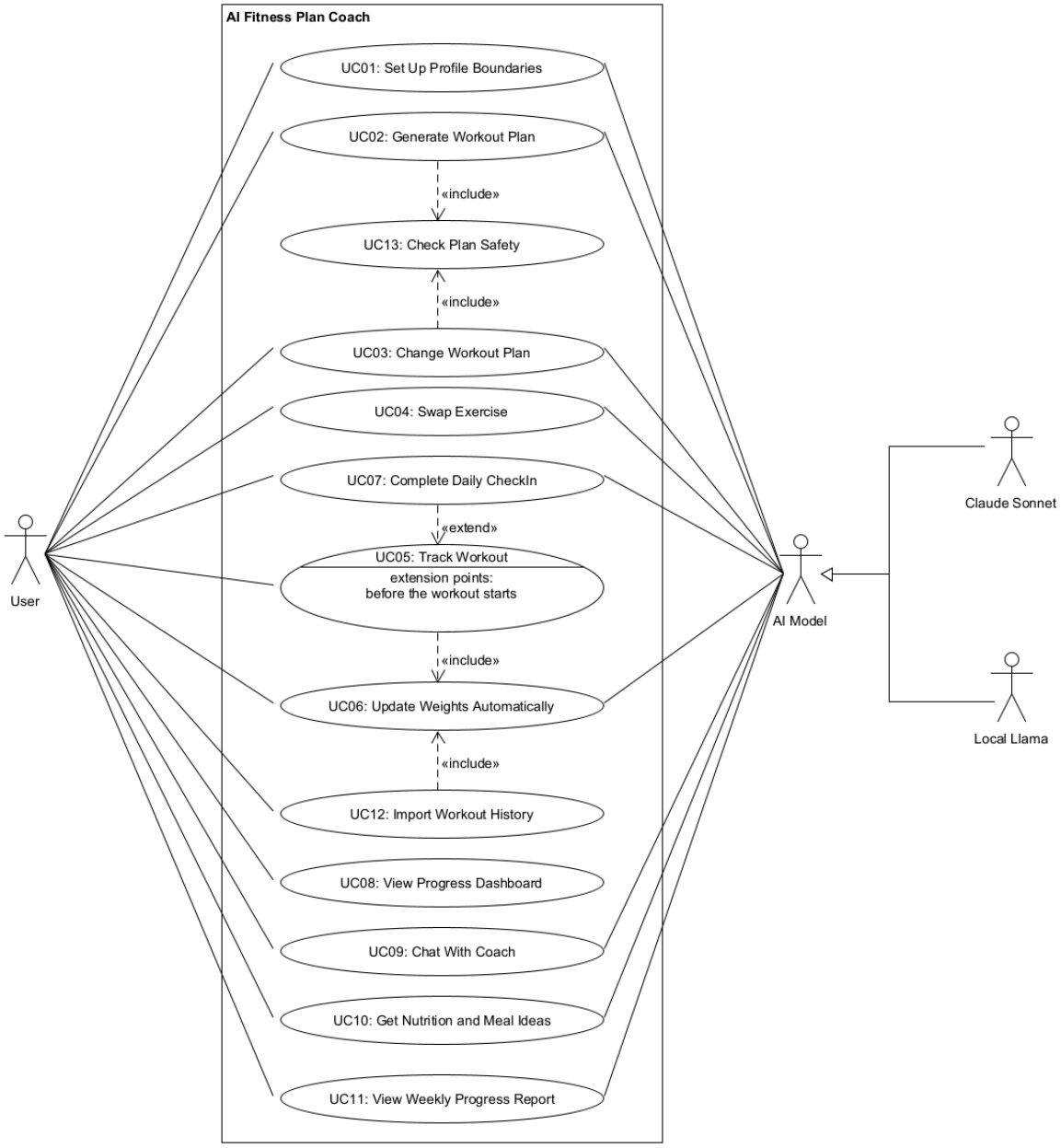
Adapter:
The adapter model is used to allow for different incompatible classes to work together and be properly integrated into the system. In the case of my app, The AIModel is the target of the adaptation with the claude and llama ai clients being adapted to fit the basic AIModel class in this project. This pattern is appropriate as they both have extremely different libraries and implementations but must be interchanged when used as an AIModel to turn their response into a readable value. It would be difficult if the pattern is not used as then all of the AI model code would be within the FitnessAgent class. This would make it difficult to update if a new version of the AI comes out and also difficult to integrate as it would be unsure which AI model is being used as claude is only used online and llama offline making the code extremely messy.

Observer:
The objective of the observer is to update information for the user in real time when data is collected. In this case when a workout is finished in the app, many other systems must immediately gain access to the completed data to update their services. For example, when a WorkoutObserver finds a new workout that was logged in WorkoutHistory, the user’s charts must update in Main and also the system must update the WeightProgression data for the next weight calculation. We need a workout observation system as there are many different systems all relying on the same observation of a new workout’s data and linking them all up to check for when a new workout is logged can be taxing to the computer and also to the readability of the code.

Strategy:
The objective of this design is to take two different algorithms that have similar inputs and outputs and choose when to use each in order to easily change the algorithms and to separate different strategies without a crazy amount of if statements. In this case, ProgressionStrategy can choose between BeginnerStrategy or IntermediateStrategy to give a weight increase that the user can use for progressive overload within WeightProgression.

Singleton:
For a workout app, we must store a library of exercises that the user’s plan can pull from. A singleton can help us with that by ensuring that there is only 1 copy of the exercise library in use rather than needlessly creating many and wasting memory. In this case, the ExerciseLibrary is the singleton and FitnessAgent, FitnessAppFacade, and DataStore all use information stored here accessing the same library as to not overuse memory or possibly corrupt data due to a difference in pointers.

2.2 Use-Case Diagram and Use-Case Descriptions



For each major use case, provide a description containing:

- **Use Case ID;**
- **Use Case Name;**
- **Actor(s);**
- **Goal;**
- **Preconditions;**
- **Trigger;**
- **Main Success Scenario;**
- **Alternative/Exception Flows;**
- **Postconditions;**
- **Related Feature(s).**

UC01 - Set Up Profile Boundaries

• **Actor(s);** Primary: The User, Secondary: AI model
• **Goal;** Saves the user's information and boundaries for their workout plans.
• **Preconditions;** The user sets up their profile through the page or command line.
• **Trigger;** The user clicks save profile.
• **Main Success Scenario;**

1. The user inputs their height, weight, age, and workout experience.
2. The user tells the AI any other determining factors such as an injury, their goal, equipment owned, or what times they are available to workout.
3. The user saves their changes.
4. The AI checks the user information to see if it is reliable.
5. The AI creates a list of boundaries for the workout plan based on user input.
6. The system asks the user to accept or decline the boundaries.
7. The user accepts the boundaries.
8. The system saves the boundaries onto the profile.

• **Alternative/Exception Flows;**

Step 4: If the information is unrealistic such as weight/height being negative or impossible asks the user to reenter.

Step 5:

a) If there is a serious medical condition listed, the system asks the user to consult a doctor first.
b) If there is an unclear detail in regards to user input then the AI will ask follow up questions.
c) If the AI is offline then the user metrics will save and the user will be asked to fill out further user information later.

• **Postconditions;** The profile is saved to the device and has a list of boundaries/information about the user.
• **Related Feature(s);** F01

UC02 - Generate Workout Plan

• **Actor(s);** Primary: User, Secondary: AI model
• **Goal;** Generate a workout plan that is contained within the boundaries the user has set and save it to the calendar.
• **Preconditions;** The user must have saved a profile
• **Trigger;** The user either uses the app or the command line to generate a plan for the number of weeks they choose.
• **Main Success Scenario;**

1. The user gives the number of weeks they would like to workout and click to generate a plan.
2. The system gives the AI model the users workout boundaries and past workout data.
3. The AI looks through the current exercise library that fits their requirements.
4. The AI builds the plan.
5. The system double checks whether all of the users boundaries were met.
6. The system allows the user to choose whether to accept the plan or to request to regenerate it.
7. The user will accept the plan.
8. The system saves the plan to the calendar.

• **Alternative/Exception Flows;**

Step 1: The user does not have a profile. The user will be redirected to the profile screen.
Step 4: The AI is offline. The plan is chosen based on a basic template.
Step 5 and 6: If there is something wrong with the plan or the user chooses to regenerate it, then a new plan will be made.
Step 8: If the user already has a plan, they will be asked if they want to replace it or discard the new one.
• **Postconditions;** The user will have a workout plan that is also saved to the calendar.
• **Related Feature(s);** F02

UC03 - Change Workout Plan
• **Actor(s);** Primary: User Secondary: AI Model
• **Goal;** Make a quick change to an already existing plan..
• **Preconditions;** User has a current plan
• **Trigger;** User types in their request and submit it or through the command line
• **Main Success Scenario;**
1. User types in a change and submits it
2. AI reads the request and finds out what exercises/dates are effected
3. AI searches for alternatives to the workouts.
4. The system will check the alternatives if they match the user boundaries.
5. The system will allow the user to either confirm, undo, or regenerate the changes to the calendar/exercise.
6. User confirms
7. The system saves the new plan and keeps a copy of the old plan in case the user wants to revert.

• **Alternative/Exception Flows;**
Step 1: If the user does not have a plan yet they will be prompted to make one.
Step 2: If the request is either unclear or unsafe then the AI might ask follow up questions or explain a safer alternative.
Step 3: This alternative workout is only for that day and will not affect next week.
Step 6: If the user regenerates then the AI will go back to searching for alternatives and if the user undos then the old plan will return.
• **Postconditions;** The new/same plan has been confirmed by the user.
• **Related Feature(s);** F03

UC04 - Swap Exercise
• **Actor(s);** Primary: User Secondary: AI Model
• **Goal;** Replace an exercise that the user does not want to do.
• **Preconditions;** The user has a workout plan
• **Trigger;** The user selects an exercise and clicks to swap the exercise.
• **Main Success Scenario;**
1. The user selects an exercise they do not want to do anymore and chooses to swap and gives the system a reason
2. The system will look for a new exercise that matches the same muscle groups.
3. The AI will pick 3 of the best options and offer them as choices for the user with reasons.
4. The system will show the 3 options
5. The user will pick an option and choose whether to replace the exercise for the rest of the plan or just today.

6. The system will update the plan.

• **Alternative/Exception Flows;**

Step 1: The user has a medical reason. Will recommend the user to go to the doctor.
Step 2: No matching exercises. The system will increase the search radius to another type of movement that still activates the muscle.
Step 3: If the AI is offline the user will not have reasoning behind the exercises but will still be recommended 3 basic exercises.

• **Postconditions;** Either today or the rest of the plan will swap exercises.
• **Related Feature(s);** F04

UC05 - Track Workout

• **Actor(s);** Primary: User
• **Goal;** Log today's workout and save it.
• **Preconditions;** The user has a plan
• **Trigger;** The user presses start/end workout
• **Main Success Scenario;**

1. The user starts their workout
2. The daily check in occurs
3. The system loads today's workout while showing target reps and sets per exercise
4. For each set the user enters the reps done and difficulty level
5. System saves the set and starts the rest timer
6. Step 4 and 5 repeat until all exercises have been completed
7. User finishes workout
8. The system saves the workout and daily summary.
9. System updates the weight for next time and updates the progress dashboard

• **Alternative/Exception Flows;**

Step 3: If no workout is planned then the user can do a free workout or start the next workout early.
Step 4:

a) If an invalid number is put for the reps then the entry will be denied and user will be asked to reenter
b) If the app is closed then the user must reopen it to continue with the past sets being saved

• **Postconditions;** The workout log is saved and dashboard and new weight limits are updated.
• **Related Feature(s);** F05

UC06 - Upgrade Weight Automatically

• **Actor(s);** Primary: AI Model Secondary: User
• **Goal;** Decides the weight limit of each exercise
• **Preconditions;** User must have a plan
• **Trigger;** When workout data is gained either from finishing a workout or importing.
• **Main Success Scenario;**

1. For each exercise the system checks whether the user hit the top rep range.
2. If it has been reached then the weight limit increases
3. If the user is making no progress or backtracking then AI is used.
4. The AI will look at recent data from workouts and check-ins and decide whether to keep the weights the same, start a deload, or change the exercise.

5. The system will show the new weight limit to the user and the reason.
6. The user presses accept

• **Alternative/Exception Flows;**
Step 1: If there is not enough data about an exercise weight stays the same.
Step 2: If the user does not have access to equipment to increase weight, reps will increase.
Step 3: If AI is offline then the weight will stay the same and wait for AI to come online.
Step 6: The user can choose to manually change their weight and the AI will remember the change.
• **Postconditions;** Plan for the exercise's weight
• **Related Feature(s);** F06

UC07 - Complete Daily Check-In
• **Actor(s);** Primary: User Secondary: AI Model
• **Goal;** Changes today's workout intensity based on user condition
• **Preconditions;** User must have a workout today
• **Trigger;** The user starts their workout
• **Main Success Scenario;**

1. The system shows the check in window asking users to rate their sleep, energy, stress, and soreness with a comment box for unique reasons.
2. User answers and submits.
3. System will turn their answer into a score and save the result
4. If the score is outside of regular ranges then the intensity of today will change with a reason given for the user.
5. The system shows the adjustment and asks the user to accept or reject.
6. The user accepts and intensity changes.

• **Alternative/Exception Flows;**
Step 2: If the user skips the daily check in then a regular workout continues.
Step 3:

a) If the comments sound serious then the user will be recommended to talk to doctors.
b) If the scores have been low for a few days then a deload will be recommended.

• **Postconditions;** The check in will be saved and intensity will be adjusted or left alone based on results.
• **Related Feature(s);** F07

UC08 - View Progress Dashboard
• **Actor(s);** Primary: User
• **Goal;** See how the users strength/body measurements change over time
• **Preconditions;** The user has some workout data
• **Trigger;** The user looks at the progress dashboard or runs a command line.
• **Main Success Scenario;**

1. The user opens the progress dashboard and chooses an exercise/muscle group/body measurement and time
2. The system calculates the stats from workout data/body data
3. The system draws and displays the charts.
4. The user can add body measurements and the chart will update.
5. The user can start a workout and the chart will update.

• **Alternative/Exception Flows;**
Step 2: The chart will not display if the user has no previous workout data

Step 3: If there is an inconsistency in the data, it will be flagged to be seen by the user.
• **Postconditions;** The chart will show the user's current data
• **Related Feature(s);** F08

UC09 - Chat With Coach
• **Actor(s);** Primary: User Secondary: AI Model
• **Goal;** Get personalized answers about user workouts, plans and any progress based on their own data.
• **Preconditions;** User has a profile
• **Trigger;** The user sends a message to the chatbox
• **Main Success Scenario;**
1. The user types a question and submits it
2. The AI will decide what information it needs to access to respond
3. The AI will search workout history, exercise library and the user's profile
4. The AI will write an answer to the user
5. The system will show the answer and sources to the relevant data that points to the conclusion in your data
6. The system will save the conversation for next time

• **Alternative/Exception Flows;**
Step 2: If the question is not on topic it will be ignored and if it is too medical it will recommend a doctor consultation.
Step 3: If the answer is not in the data then the bot will say that an answer could not be found rather than hallucinating.
• **Postconditions;** The user receives an answer from the AI bot and the conversation is saved
• **Related Feature(s);** F09

UC10 - Get Nutrition and Meal Ideas
• **Actor(s);** Primary: User Secondary: AI Model
• **Goal;** Calculate daily calories and macronutrients and come up with meals following this logic and any food restrictions for a user.
• **Preconditions;** User profile is complete and body measurements are in
• **Trigger;** User presses calculate target.
• **Main Success Scenario;**
1. The user presses calculate target
2. The system calculates daily calories and macronutrient target for the plan
3. User enters food preferences and restrictions
4. AI creates meal ideas and make sure that they fit the macronutrient target
5. System double checks the food restrictions
6. System asks user whether to accept the meal or regenerate
7. User accepts and the system remembers their preferences

• **Alternative/Exception Flows;**
Step 2: If their information is missing they will be prompted to fill that in and if their target calories is too low then it will be automatically heightened to a safe degree.
Step 4: If the AI is offline then meal ideas are disabled.
Step 5 &7: If a meal breaks the restriction or the user chooses to regenerate then a new meal idea is made.

• **Postconditions;** Target nutrition is shown and any accepted meals or food restrictions/preferences are saved.
• **Related Feature(s);** F10

UC11 - View Weekly Progress Report
• **Actor(s);** Primary: User Secondary: AI Model
• **Goal;** Create a report for the week and make recommendations for next week
• **Preconditions;** The user has a plan
• **Trigger;** The end of a week or if the user requests one
• **Main Success Scenario;**
1. The system calculates this week's statistics
2. AI collects weekly workout and daily check in data
3. AI compares it to the users goals and last week's suggestions
4. AI writes a short review on a users week and suggestions for next week
5. The system shows the report and asks user whether to accept or deny the next week suggestions
6. The user accepts the suggestions and the plan is changed
7. The user saves the report

• **Alternative/Exception Flows;**
Step 2: If there was no workout this week then the suggestions will be targeted towards getting back on track
Step 4: If the AI is offline only data comparison will be available in the report.
Step 6: Weekly plan will continue as normal with no changes.
Step 7: If the report is not saved then it is automatically discarded and cannot be retrieved.
• **Postconditions;** The user reviews the weekly report and chooses whether to change their plan or to save it
• **Related Feature(s);** F11, F03

UC12 - Import Workout History
• **Actor(s);** Primary: User
• **Goal;** Imports any user workout data from other apps
• **Preconditions;** User has a CSV file exported from another program
• **Trigger;** User clicks to import the file
• **Main Success Scenario;**
1. The user chooses which CSV file to import
2. The system reads the file and tries to find what format it is in
3. The system matches the exercises in the file to the one in its own library
4. The system shows what exercises it could not match
5. The user matches those exercises manually then clicks to import
6. The system adds the workout data to the user's past history and shows a summary of the imported data
7. The progressive weight and progress dashboard is refreshed with the new data

• **Alternative/Exception Flows;**
Step 2:
a) If the file type is wrong or the data is corrupted then it is ignored and throws an error to the user
b) If the weight is in pounds it is automatically switched to kilograms

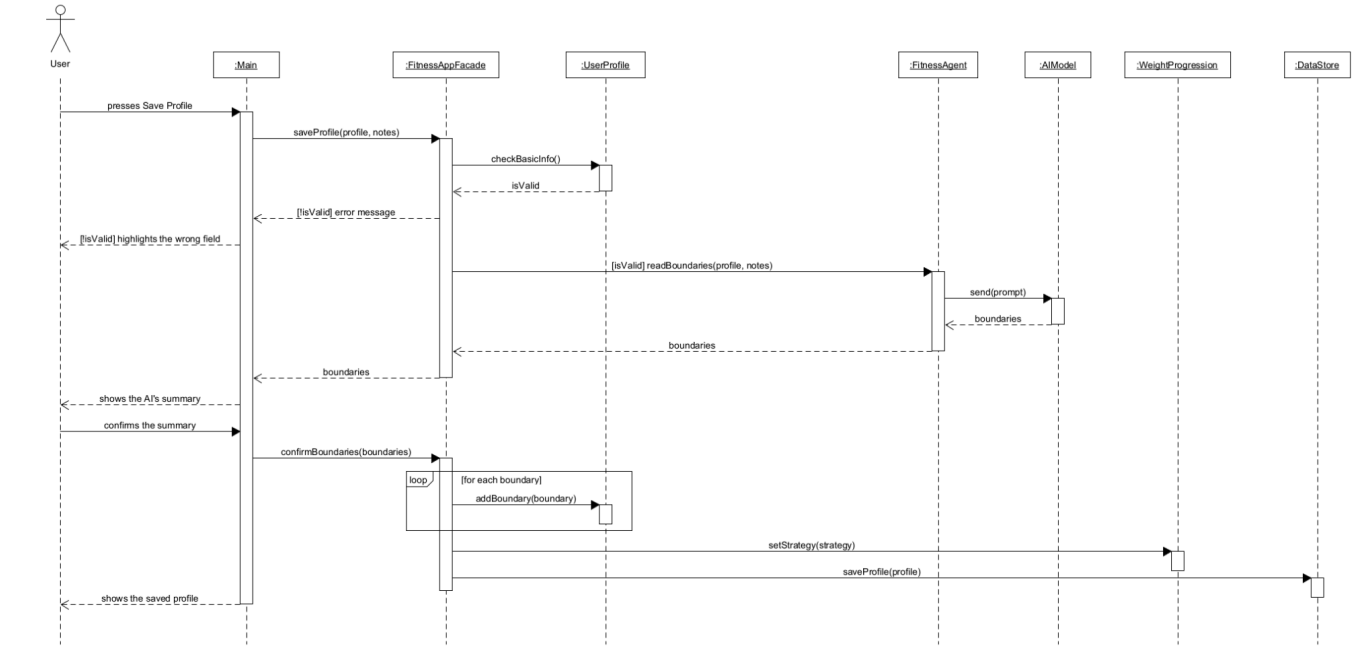
Step 6: If some workouts were already imported before then the app skips the duplicates.

• **Postconditions;** The workout data is imported and properly formatted
• **Related Feature(s);** F12, F06, F08

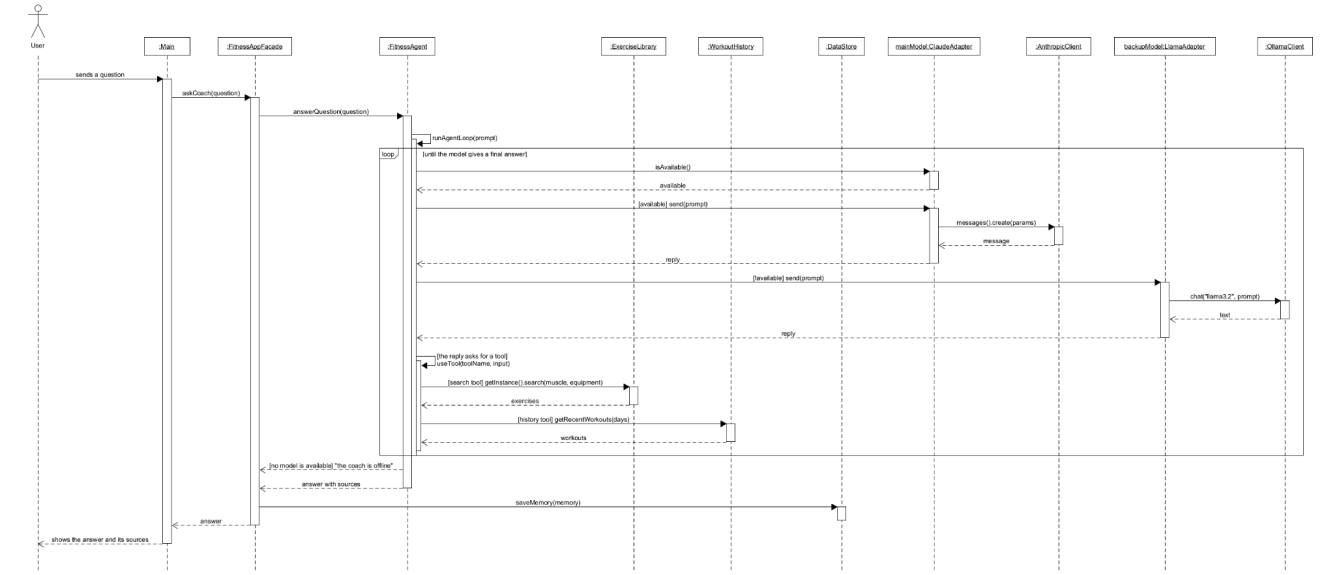
UC13 - Check Plan Safety
• **Actor(s);** None.
• **Goal;** Make sure that the AI's plan follows current user boundaries
• **Preconditions;** AI model just made a new plan
• **Trigger;** UC02 and UC03 changed the user plan
• **Main Success Scenario;**
    1. The system checks if the exercise can strain any of the user's injuries
    2. The system checks if the exercise fits the times the user is available within
    3. The system checks if there is any over/under strained muscle groups
    4. If no problems are found then the plan will go back and be given to the user
• **Alternative/Exception Flows;**
Step 4: If any problems are found, rerun the plan builder and recheck. If the recheck still has an error after 3 checks then give the user the plan with a warning sign.
• **Postconditions;** The plan follows the user boundaries or is delivered with a warning sign
• **Related Feature(s);** F02, F03

2.3 Sequence Diagrams

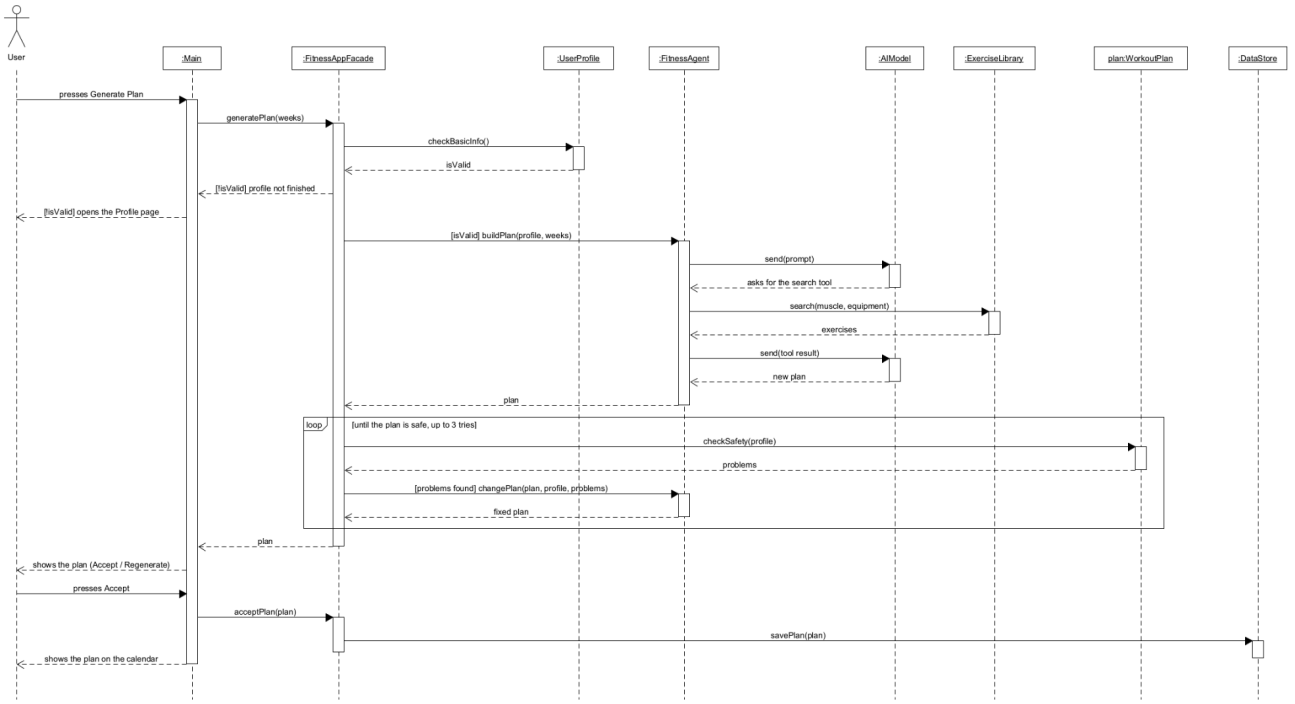
SD01 - Set Up Profile Boundaries (F01)



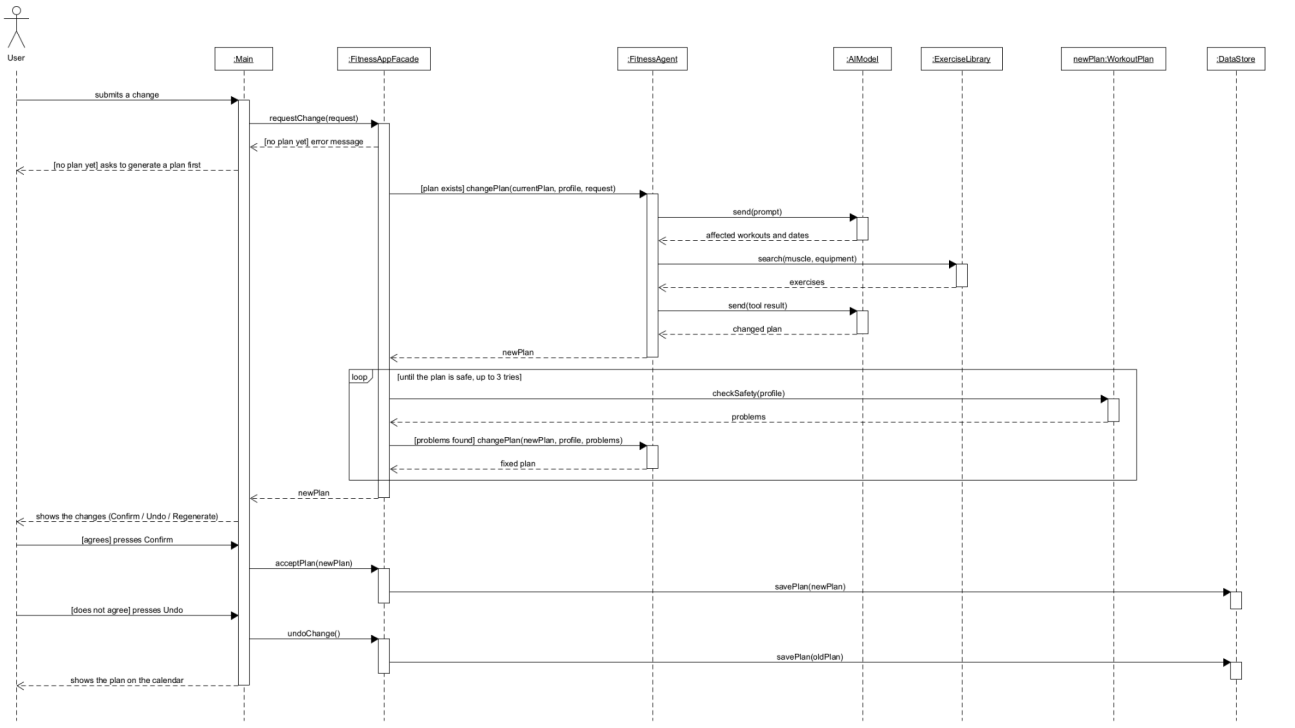
SD02 - Coach Chatbox and the AI Agent Loop (F09)



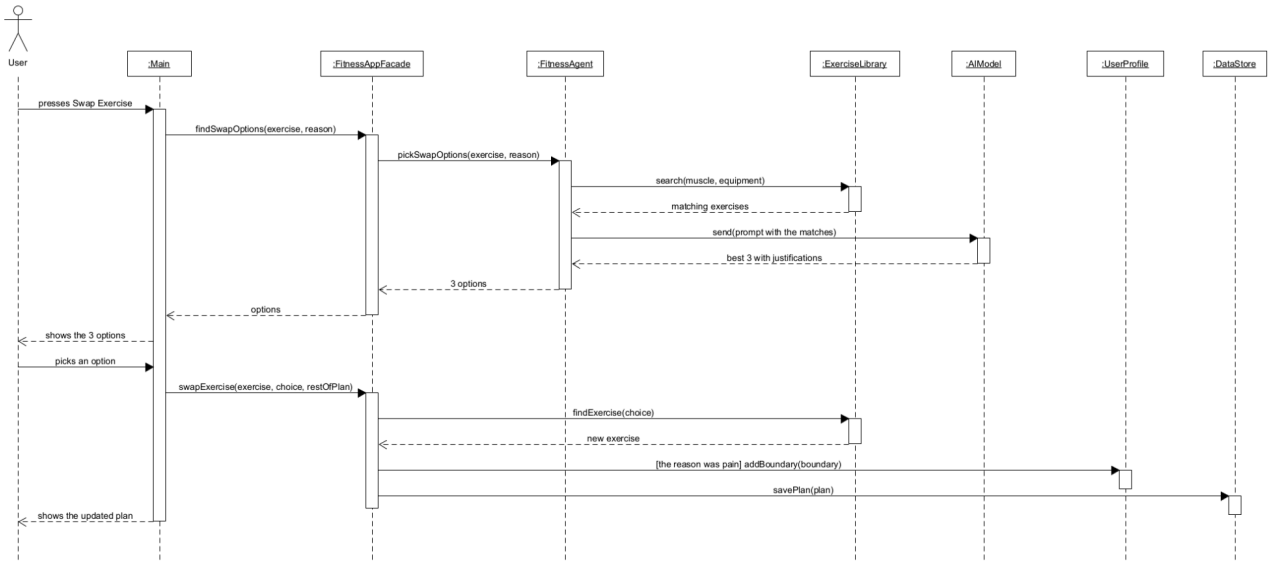
SD03 - Generate Workout Plan (F02)



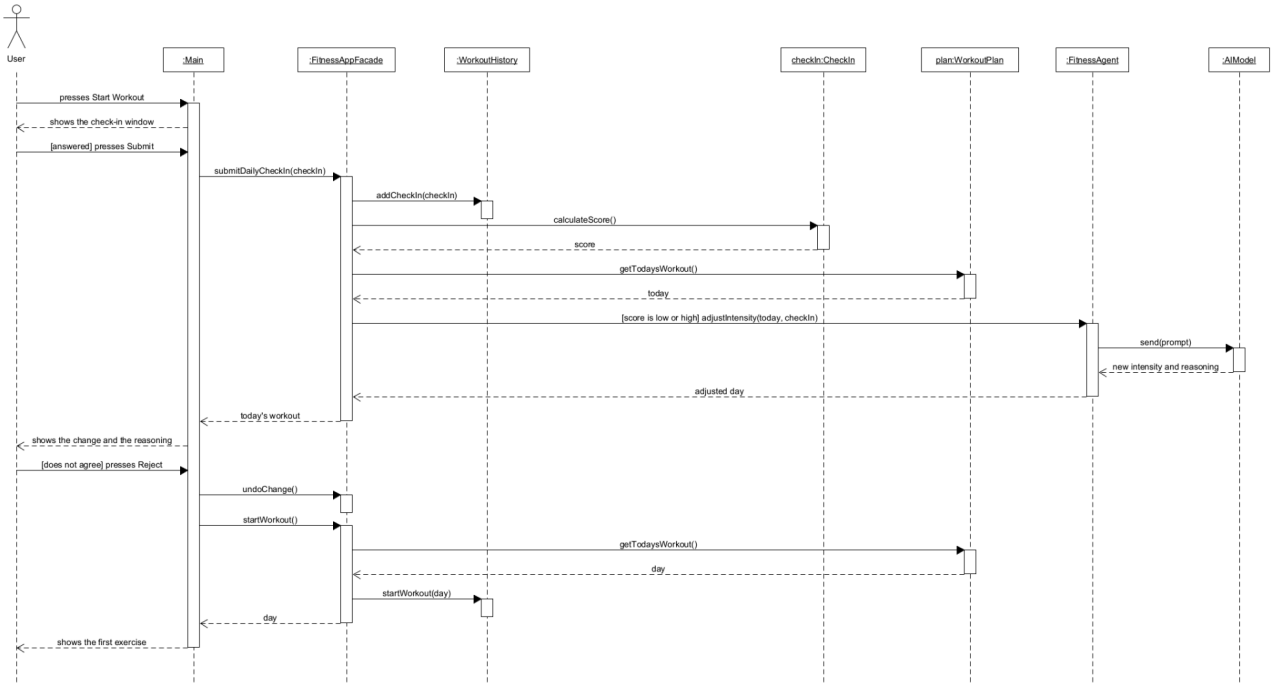
SD04 - Change the Workout Plan and Undo (F03)



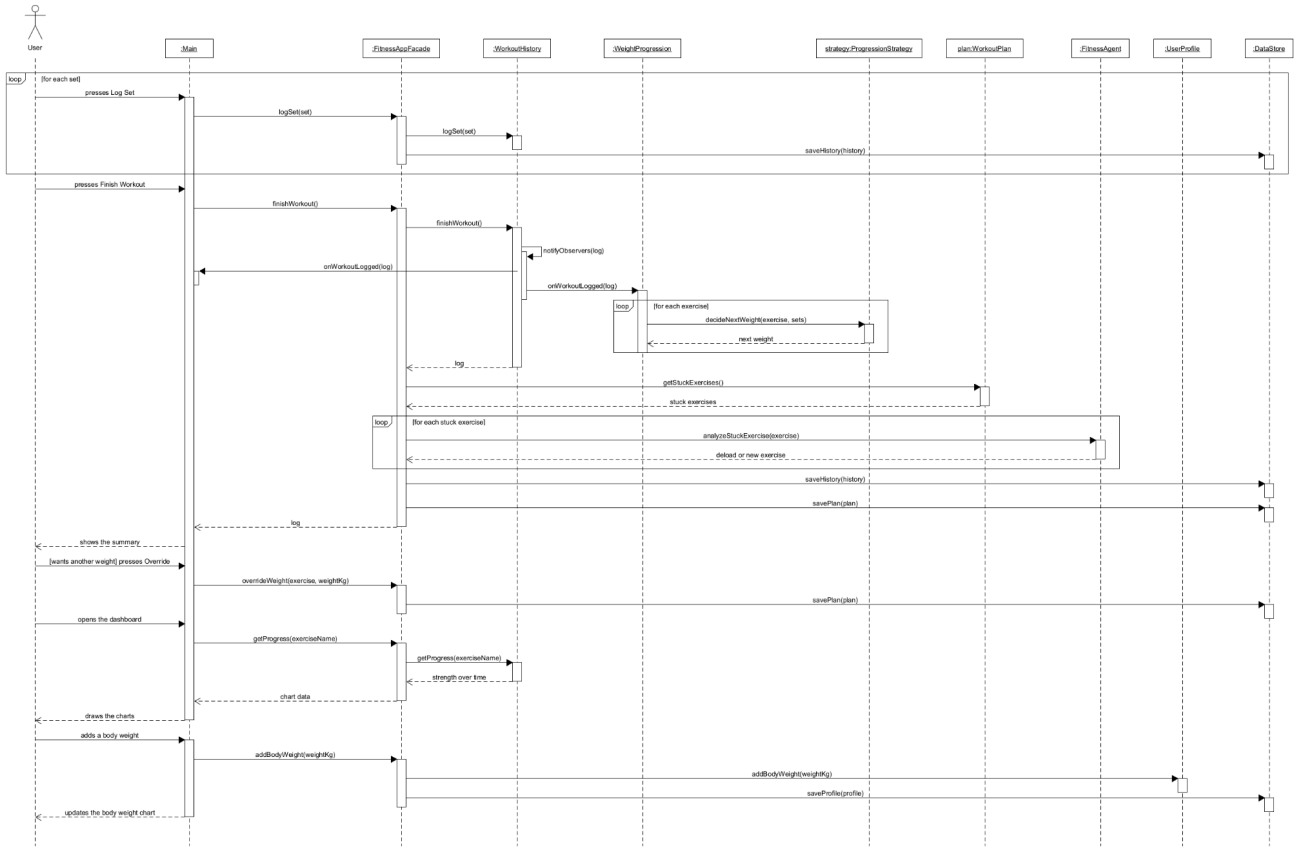
SD05 - Swap Exercise (F04)



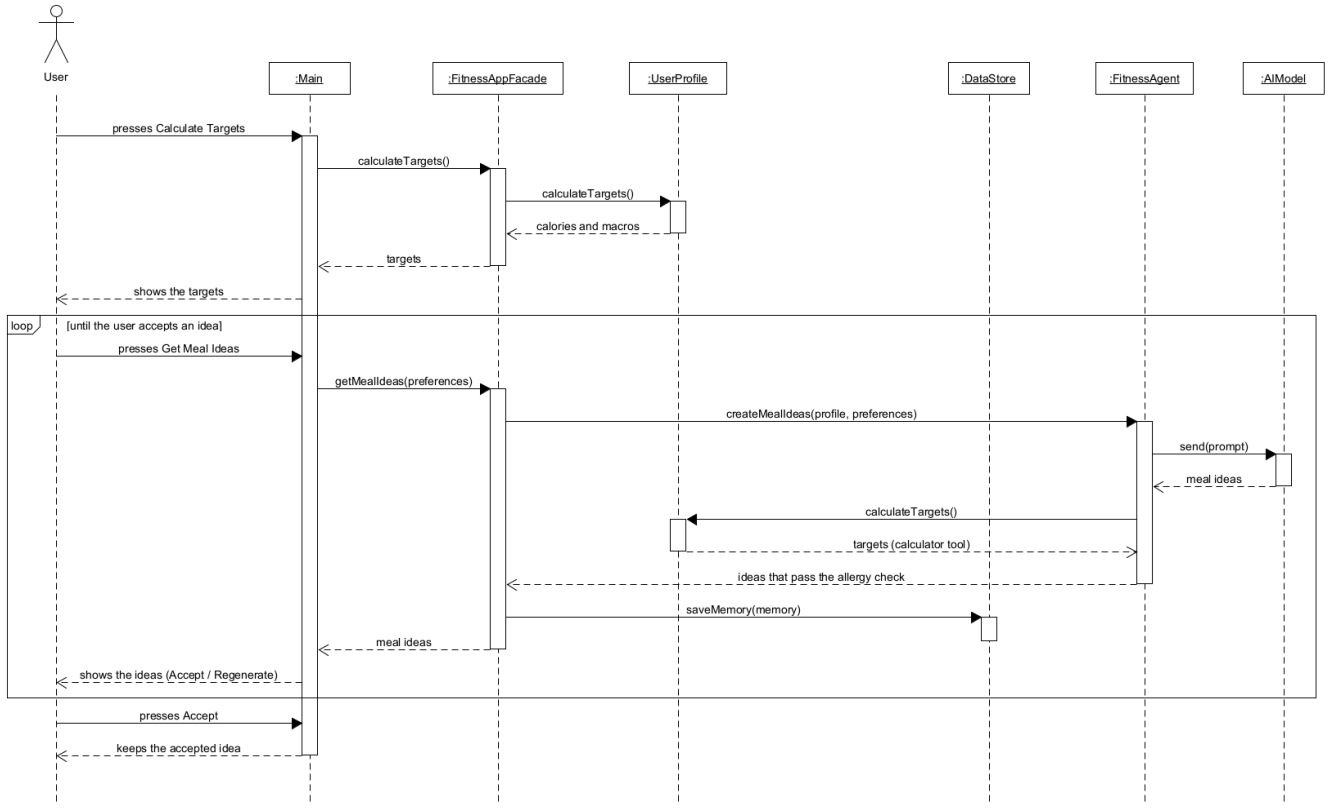
SD06 - Daily CheckIn and Start Workout (F07, F05)



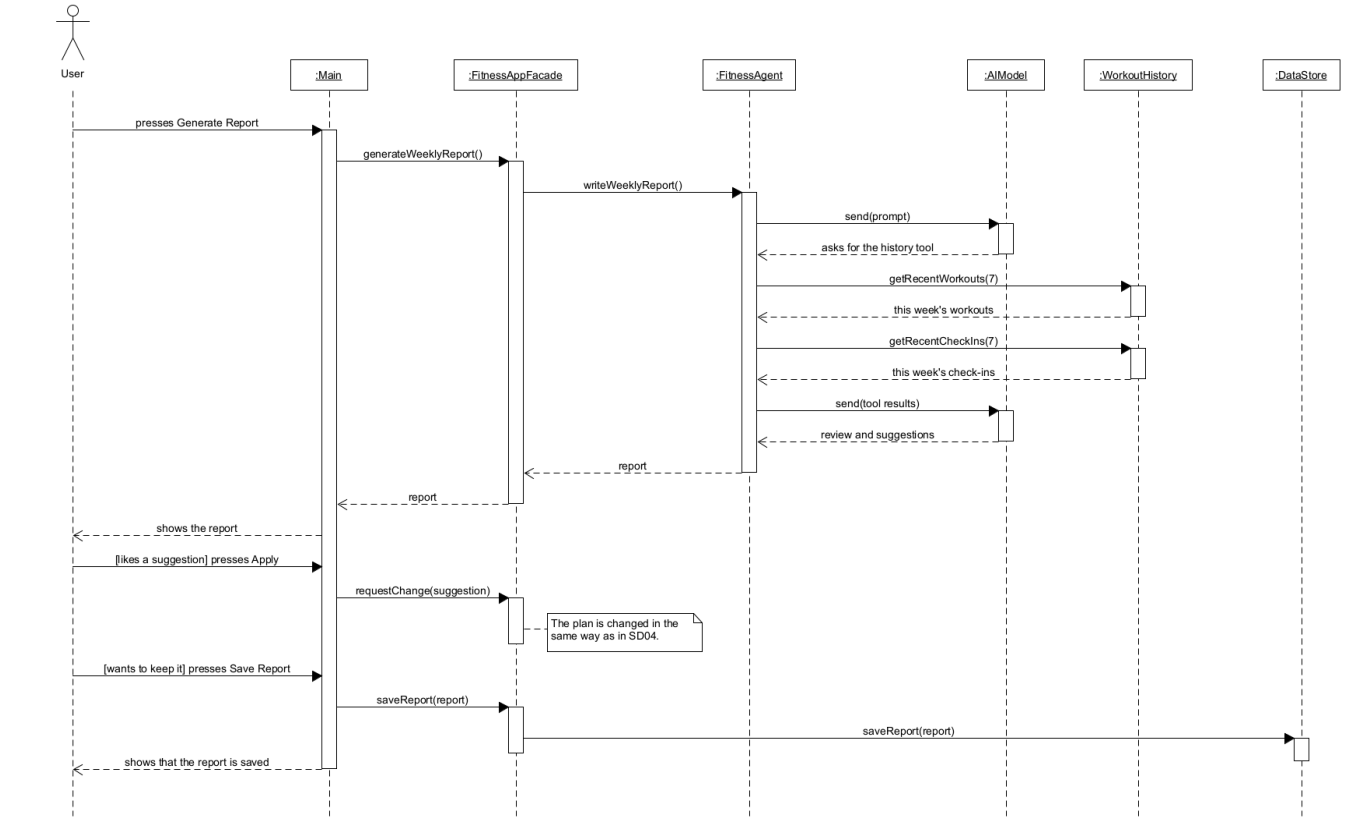
SD07 - Log Sets, Finish Workout, Weight Progression and Dashboard (F05, F06, F08)



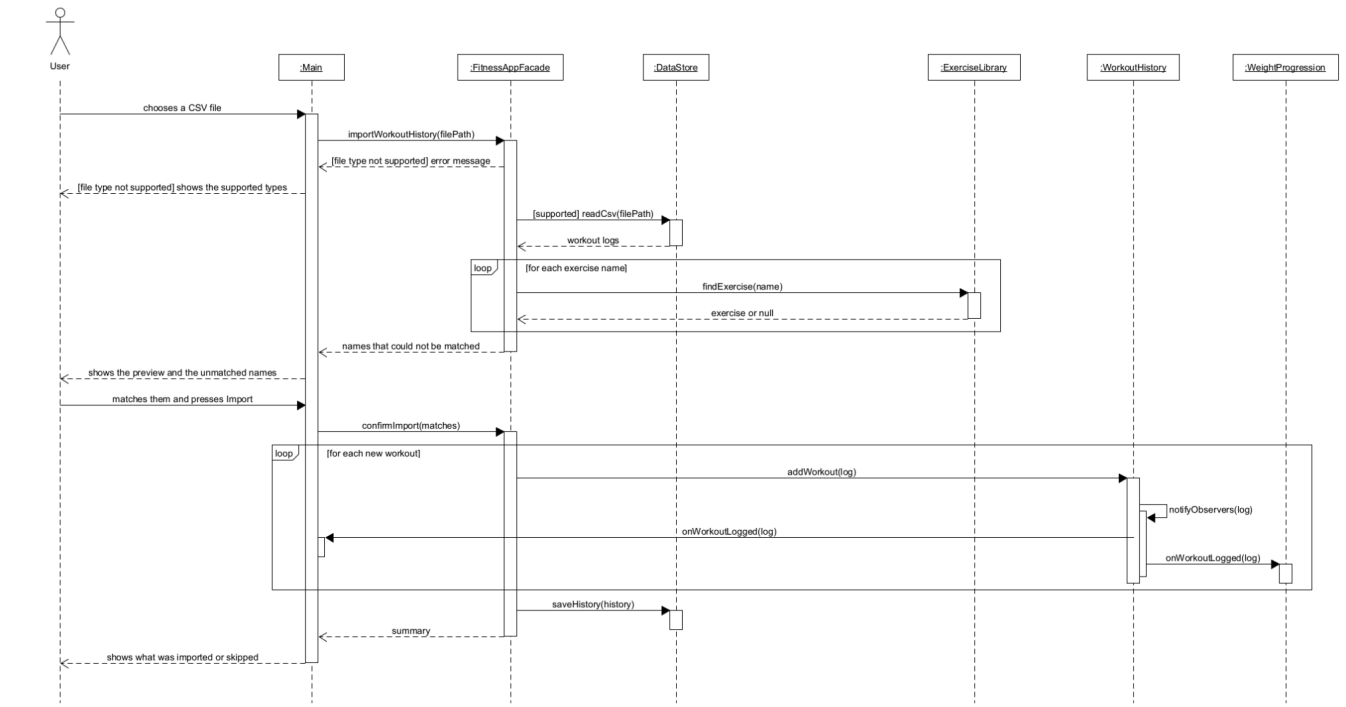
SD08 - Nutrition and Meal Ideas (F10)



SD09 - Weekly Progress Report (F11)



SD10 - Import Workout History (F12)



Task 3: Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01 - User Profile Boundaries	Saves user info and AI makes a list of boundaries based on requests.	Hybrid	UC01 - Set Up Profile Boundaries	Main, FitnessAppFacade, UserProfile, FitnessAgent, AIModel, WeightProgression, DataStore	saveProfile(), checkBasicInfo(), readBoundaries(), send(), confirmBoundaries(), addBoundary(), setStrategy()	SD01	Facade, Adapter, Strategy
F02 - Workout Plan Generation	Make a workout plan that follows previous user boundaries.	AI-based	UC02 - Generate Workout Plan, UC13 - Check Plan Safety	Main, FitnessAppFacade, UserProfile, FitnessAgent, AIModel, Exercise Library, Workout Plan, Workout Day, Planned Exercise, DataStore	generatePlan(), checkBasicInfo(), buildPlan(), send(), search(), checkSafety(), changePlan(), acceptPlan(), savePlan()	SD03	Facade, Adapter, Singleton
F03 - User Defined Changes To The Workout Plan	Changes workout plan based on user request. Once confirmed cannot be undone.	AI-based	UC03 - Change Workout Plan, UC13 - Check Plan Safety	Main, FitnessAppFacade, FitnessAgent, AIModel, Exercise Library, Workout Plan, DataStore	requestChange(), changePlan(), send(), search(), checkSafety(), acceptPlan(), undoChange(), savePlan()	SD04	Facade, Adapter, Singleton

F04 - Exercise Swapping	Changes an exercise user declines with one of the three picked by AI	Hybrid	UC04 - Swap Exercise	Main, FitnessAppFacade, FitnessAgent, AIModel, ExerciseLibrary, Exercise, PlannedExercise, UserProfile, DataStore	findSwapOptions(), pickSwapOptions(), search(), send(), swapExercise(), findExercise(), addBoundary(), savePlan()	SD05	Facade, Adapter, Singleton
F05 - Workout Tracker	Track each day's workout.	Deterministic	UC05 - Track Workout	Main, FitnessAppFacade, WorkoutPlan, WorkoutDay, WorkoutHistory, WorkoutLog, SetLog, WorkoutObserver, DataStore	startWorkout(), getTodaysWorkout(), logSet(), addSet(), saveHistory(), finishWorkout(), getWeightLifted(), notifyObservers()	SD06, SD07	Facade, Observer
F06 - Automatic Weight Progression	Chooses the next exercise weight range based on user data. If data shows a downward trend, it allows for AI to intervene.	Hybrid	UC06 - Update Weights Automatically	Main, FitnessAppFacade, WorkoutHistory, WorkoutObserver, WeightProgression, ProgressionStrategy, BeginnerStrategy, Intermedi	onWorkoutLogged(), decideNextWeight(), getStuckExercises(), analyzeStuckExercise(), send(), savePlan(), overrideWeight()	SD07	Facade, Adapter, Observer, Strategy

				ateStrategy, WorkoutPlan, PlannedExercise, FitnessAgent, AIModel, DataStore			
F07 - Daily CheckIn	Gets user input based on health markers and adjusts intensity of today's workout.	Hybrid	UC07 - Complete Daily CheckIn	Main, FitnessAppFacade, CheckIn, WorkoutHistory, WorkoutPlan, WorkoutDay, FitnessAgent, AIModel	submitDailyCheckIn(), addCheckIn(), calculateScore(), getTodaysWorkout(), adjustIntensity(), send(), undoChange()	SD06	Facade, Adapter
F08 - Progress Dashboard	Shows user's strength and body weight on a chart that updates after each workout.	Deterministic	UC08 - View Progress Dashboard	Main, FitnessAppFacade, WorkoutHistory, WorkoutLog, WorkoutObserver, UserProfile, DataStore	addObserver(), onWorkoutLogged(), getProgress(), addBodyWeight(), saveProfile()	SD07	Facade, Observer
F09 - Coach Chatbox	Answers questions that users may have about their data/plan.	AI-based	UC09 - Chat With Coach	Main, FitnessAppFacade, FitnessAgent, AIModel, ClaudeAdapter, LlamaAd	askCoach(), answerQuestion(), runAgentLoop(), isAvailable(), send(), useTool(),	SD02	Facade, Adapter, Singleton

				apter, ExerciseLibrary, WorkoutHistory, DataStore	search(), getRecentWorkouts(), saveMemory()		
F10 - Nutrition and Meals	Calculates the amount of calories and macronutrients needed. AI can suggest meals that fit the range.	Hybrid	UC10 - Get Nutrition and Meal Ideas	Main, FitnessAppFacade, UserProfile, FitnessAgent, AIModel, DataStore	calculateTargets(), getMealIdeas(), createMealIdeas(), send(), saveMemory()	SD08	Facade, Adapter
F11 - Weekly Progress Report	Ai gives users a weekly report and can suggest changes for upcoming weeks.	Ai-based	UC11 - View Weekly Progress Report	Main, FitnessAppFacade, FitnessAgent, AIModel, WorkoutHistory, DataStore	generateWeeklyReport(), writeWeeklyReport(), send(), getRecentWorkouts(), getRecentCheckIns(), requestChange(), saveReport()	SD09	Facade, Adapter
F12 - Import Workout History	Import workout history from other apps.	Deterministic	UC12 - Import Workout History	Main, FitnessAppFacade, DataStore, ExerciseLibrary, WorkoutHistory, WorkoutLog, WorkoutObserver,WeightProgression	importWorkoutHistory(), readCsv(), findExercise(), confirmImport(), addWorkout(), notifyObservers(), saveHistory()	SD10	Facade, Observer, Singleton

Task 4: Explain How Each Feature Is Realized

F01 - User Profile Boundaries
Related Use Case: UC01 - Set Up Profile Boundaries
Related Sequence Diagram: SD01 - Set Up Profile Boundaries
Classes involved:

- Main: Collects the user's information and shows the AI's summary.
- FitnessAppFacade: coordinates the request.
- UserProfile: holds the user's boundary information and checks the basic information.
- FitnessAgent: turns the notes into a list of boundaries.
- AIModel: AI model reads the notes.
- WeightProgression: gets the weight progression that fits the user's past experience.
- DataStore: saves the profile.

Important methods:

- FitnessAppFacade.saveProfile()
- UserProfile.checkBasicInfo()
- FitnessAgent.readBoundaries()
- AIModel.send()
- FitnessAppFacade.confirmBoundaries()
- UserProfile.addBoundary()
- WeightProgression.setStrategy()
- DataStore.saveProfile()

Execution:
When the user saves their profile, Main calls for FitnessAppFacade.saveProfile() which checks UserProfile.checkBasicInfo(). If the entered information is not valid then Main tells the user what fields are invalid. If it is valid, FitnessAgent.readBoundaries() sends the notes to the ai using AIModel.send() which gives back the boundary list which Main then confirms with the user. If the user confirms it, FitnessAppFacade.confirmBoundaries() adds the boundaries to UserProfile.addBoundary(), picks how the weight should progress in the plan using WeightProgression.setStrategy() then finally saves it with DataStore.saveProfile().

F02 - Workout Plan Generation
Related Use Case: UC02 - Generate Workout Plan, UC13 - Check Plan Safety
Related Sequence Diagram: SD03 - Generate Workout Plan
Classes involved:

- Main:GUI is the Workout Plan tab. It takes a number of weeks and gives you a plan with either Accept or Regenerate options.
- FitnessAppFacade: coordinates the request and runs the safety check.
- UserProfile: gives the user assigned boundaries that the plan has to follow.
- FitnessAgent: builds the plan and fixes it if the safety check finds any problems.
- AIModel: writes the plan.
- ExerciseLibrary: A Singleton used by the AI agents with a library of exercises.
- WorkoutPlan: the new plan. checkSafety() is the double check from UC13.
- WorkoutDay & PlannedExercise: the days of the plan and the exercises used in each day with their sets, reps and weight.
- DataStore: it saves the accepted plan.

Important methods:

- FitnessAppFacade.generatePlan()

- UserProfile.checkBasicInfo()
- FitnessAgent.buildPlan()
- AIModel.send()
- ExerciseLibrary.search()
- WorkoutPlan.checkSafety()
- FitnessAgent.changePlan()
- FitnessAppFacade.acceptPlan()
- DataStore.savePlan()

Execution:
When the user generates a plan with the GUI, Main will call on FitnessAppFacade.generatePlan(). If UserProfile.checkBasicInfo() is not complete then the profile will first be asked to be finished. Otherwise, FitnessAgent.buildPlan() will run instead and ask the AI to create a plan with AIModel.send(). The AI will then access ExerciseLibrary.search() to look for different exercises that fit the user's conditions. This plan is then double checked with WorkoutPlan.checkSafety() and any problems will make it go to FitnessAgent.changePlan() at most 3 times before a warning is used instead. When the user accepts the plan with FitnessAppFacade.acceptPlan(), the new one will then be saved with DataStore.savePlan().

F03 - User Defined Changes To The Workout Plan
Related Use Case: UC03 - Change Workout Plan, UC13 - Check Plan Safety
Related Sequence Diagram: SD04 - Change the Workout Plan and Undo
Classes involved:

- Main: GUI is the Workout Plan tab. Takes the user request and shows the changes with the option to Confirm, Undo and Regenerate plans.
- FitnessAppFacade: coordinates the request and keeps the old plans in its oldPlans stack just in case of an undo.
- FitnessAgent: changes only the affected workouts.
- AIModel: finds which workouts and dates the request affects.
- ExerciseLibrary: the search tool to find replacement exercises.
- WorkoutPlan: the new optional plan, double checked with checkSafety().
- DataStore: saves the new plan or old plan after an undo.

Important methods:

- FitnessAppFacade.requestChange()
- FitnessAgent.changePlan()
- AIModel.send()
- ExerciseLibrary.search()
- WorkoutPlan.checkSafety()
- FitnessAppFacade.acceptPlan()
- FitnessAppFacade.undoChange()
- DataStore.savePlan()

Execution:
A user submits a request to change the workout plan from Main through FitnessAppFacade.requestChange(). If there is no current plan then the user is asked to have an active plan to make further changes. If there is one, FitnessAgent.changePlan() sends a request to the AI with AIModel.send() to find the affected workouts and to replace them with other workouts in ExerciseLibrary.search(). The new plan is then checked for boundaries with WorkoutPlan.checkSafety() and then the user is asked to make a choice.

Accepting the new plan calls FitnessAppFacade.acceptPlan() which saves the new plan to DataStore.savePlan(). Undoing this plan after that calls FitnessAppFacade.undoChange() which brings back the old plan.

F04 - Exercise Swapping
Related Use Case: UC04 - Swap Exercise
Related Sequence Diagram: SD05 - Swap Exercise
Classes involved:

- Main: GUI is the Workout tab. Shows the three options to swap exercises with and an option for it to be Today Only or the Rest of the Plan.
- FitnessAppFacade: coordinates the exercise swap.
- FitnessAgent: finds the swap options.
- AIModel: picks the best three options and justifies each exercise.
- ExerciseLibrary: the search tool to find replacement exercises.
- Exercise: an exercise from the library, with a list of alternatives that can replace it.
- PlannedExercise: the exercise in the plan that is getting swapped.
- UserProfile: gets a new boundary if the reason for the swap was injury.
- DataStore: saves the updated plan.

Important methods:

- FitnessAppFacade.findSwapOptions()
- FitnessAgent.pickSwapOptions()
- ExerciseLibrary.search()
- AIModel.send()
- FitnessAppFacade.swapExercise()
- ExerciseLibrary.findExercise()
- UserProfile.addBoundary()
- DataStore.savePlan()

Execution:
When a user chooses to swap one of their exercises, Main will call FitnessAppFacade.findSwapOptions() to search for optional exercises to swap the current one with FitnessAgent.pickSwapOptions(). The chosen exercises will contribute to the growth of the same muscle group when found in ExerciseLibrary.search() with the AI choosing the best 3 with reasoning using AIModel.send(). When a user picks an exercise, it must be swapped in with FitnessAppFacade.swapExercise() through the ExerciseLibrary.findExercise(). If the reason for switching the exercise was injury, it must be added to the user boundaries through UserProfile.addBoundary(). They also get the option to choose if it is just for today or for the rest of the plan which is saved through DataStore.savePlan().

F05 - Workout Tracker
Related Use Case: UC05 - Track Workout
Related Sequence Diagram: SD06 - Daily CheckIn and Start Workout, SD07 - Log Sets, Finish Workout, Weight Progression and Dashboard
Classes involved:

- Main:GUI is the Workout tab with Start Workout, Log Set, and Finish Workout.
- FitnessAppFacade: passes data to the workout history.

- WorkoutPlan: gives today's workout from the workout plan.
- WorkoutDay: today's planned workout.
- WorkoutHistory: Has the workout in progress and tells the observers when it is finished.
- WorkoutLog: one workout, made up of all the logged sets.
- SetLog: one logged set, with the reps, weight and difficulty.
- WorkoutObserver: the interface for the classes that are told about a finished workout.
- DataStore: saves the history.

Important methods:

- FitnessAppFacade.startWorkout()
- WorkoutPlan.getTodaysWorkout()
- WorkoutHistory.startWorkout()
- FitnessAppFacade.logSet()
- WorkoutHistory.logSet()
- WorkoutLog.addSet()
- DataStore.saveHistory()
- FitnessAppFacade.finishWorkout()
- WorkoutHistory.finishWorkout()
- WorkoutLog.getWeightLifted()
- WorkoutHistory.notifyObservers()

Execution:

When the user starts their workout, Main will call FitnessAppFacade.startWorkout() which starts the daily workout that is under the plan in WorkoutPlan.getTodaysWorkout() and starts a new log under WorkoutHistory.startWorkout(). When a set is therefore logged, it is passed from FitnessAppFacade.logSet() towards WorkoutHistory.logSet() which adds it to the workout history under WorkoutLog.addSet() and is saved immediately using DataStore.saveHistory() in case the app crashes. When the user is done with the workout, FitnessAppFacade.finishWorkout() calls for WorkoutHistory.finishWorkout() in order to officially label the workout as completed and gives the user a summary of the workout through WorkoutLog.getWeightLifted(). Finally WorkoutHistory.notifyObservers() notifies to update the dashboard GUI and updates the charts and other data.

F06 - Automatic Weight Progression

Related Use Case: UC06 - Update Weights Automatically

Related Sequence Diagram: SD07 - Log Sets, Finish Workout, Weight Progression and Dashboard

Classes involved:

- Main: GUI is the Workout Plan tab. Shows the new weights with the option to Accept or Override them.
- FitnessAppFacade: coordinates the request and sends the stuck exercises to the agent.
- WorkoutHistory: holds all of the user's workouts and tells the observers when a new one is added.
- WorkoutObserver: the interface for the classes that get told when a new workout is added.
- WeightProgression: works out the next weights when a new workout is added.
- ProgressionStrategy: the abstract strategy that decides the next weight.

- BeginnerStrategy & IntermediateStrategy: the two weight rules, one for beginners and one for intermediate lifters.
- WorkoutPlan: finds the stuck exercises in the plan.
- PlannedExercise: the exercise in the plan that is getting a new weight.
- FitnessAgent: sends each stuck exercise to the AI.
- AIModel: decides if a stuck exercise needs a deload or a new exercise.
- DataStore: saves the updated plan.

Important methods:

- WeightProgression.onWorkoutLogged()
- ProgressionStrategy.decideNextWeight()
- WorkoutPlan.getStuckExercises()
- FitnessAgent.analyzeStuckExercise()
- AIModel.send()
- DataStore.savePlan()
- FitnessAppFacade.overrideWeight()

Execution:

When a workout is finished/imported WeightProgression.onWorkoutLogged() will run by itself since it is one of the observers of the workout history and will decide whether or not to shift the weight by looking at each exercise. For each exercise it calls ProgressionStrategy.decideNextWeight() which uses either the beginner or intermediate rule. After a finished workout, FitnessAppFacade also checks for stuck exercises with WorkoutPlan.getStuckExercises() and sends them to FitnessAgent.analyzeStuckExercise() which asks the AI with AIModel.send() if they need a deload or a new exercise as they are exercises in which the user has not significantly progressed in a long time. The new weights are then saved with DataStore.savePlan() to be used in later exercises. If the user does not like a new weight then they can put in their own weight through FitnessAppFacade.overrideWeight().

F07 - Daily Check-In

Related Use Case: UC07 - Complete Daily CheckIn

Related Sequence Diagram: SD06 - Daily CheckIn and Start Workout

Classes involved:

- Main: GUI is the Workout tab. Shows the check-in window and the changed workout with the option to Accept or Reject it.
- FitnessAppFacade: coordinates the request.
- CheckIn: the user's check-in answers, which get turned into a score.
- WorkoutHistory: keeps every check-in so they can be used later
- WorkoutPlan: gives today's workout.
- WorkoutDay: today's workout with its exercises.
- FitnessAgent: changes the intensity of today's workout.
- AIModel: decides if the workout should be easier or harder and explains why.

Important methods:

- FitnessAppFacade.submitDailyCheckIn()
- WorkoutHistory.addCheckIn()
- CheckIn.calculateScore()
- WorkoutPlan.getTodaysWorkout()
- FitnessAgent.adjustIntensity()
- AIModel.send()

- FitnessAppFacade.undoChange()

Execution:
When the user starts a workout, Main will first show the daily check-in GUI. If the user submits it, FitnessAppFacade.submitDailyCheckIn() saves the answers with WorkoutHistory.addCheckIn() and turns the answers into a score with CheckIn.calculateScore(). If the score is too low or high then the workout from WorkoutPlan.getTodaysWorkout() gets its intensity changed through FitnessAgent.adjustIntensity() which asks the AI with AIModel.send() to make it easier or harder and to explain why. If the user rejects the change, FitnessAppFacade.undoChange() brings back the normal workout. If the user skips the check-in then the workout starts like normal.

F08 - Progress Dashboard
Related Use Case: UC08 - View Progress Dashboard
Related Sequence Diagram: SD07 - Log Sets, Finish Workout, Weight Progression and Dashboard
Classes involved:
- Main: GUI is the Progress Dashboard tab. Draws the charts and gets told when a new workout is added.
- FitnessAppFacade: coordinates the request.
- WorkoutHistory: holds all of the user's workouts and tells the observers when a new one is added. It also works out the progress for the charts.
- WorkoutLog: the record of one workout with its logged sets.
- WorkoutObserver: the interface for the classes that get told when a new workout is added.
- UserProfile: keeps the user's body weight history.
- DataStore: saves the profile.

Important methods:
- WorkoutHistory.addObserver()
- Main.onWorkoutLogged()
- FitnessAppFacade.getProgress()
- WorkoutHistory.getProgress()
- FitnessAppFacade.addBodyWeight()
- UserProfile.addBodyWeight()
- DataStore.saveProfile()

Execution:
Main is added to the workout history's observers with WorkoutHistory.addObserver() so whenever a workout is finished or imported, Main.onWorkoutLogged() will refresh the charts by itself in order to display them to users on the Progress Dashboard. When the user opens the dashboard, Main calls FitnessAppFacade.getProgress() which retrieves the user's strength over time from WorkoutHistory.getProgress() and then Main draws the charts. If the user adds their body weight, FitnessAppFacade.addBodyWeight() adds it to the profile with UserProfile.addBodyWeight() and then saves it with DataStore.saveProfile() so the body weight chart is updated and can be analyzed too.

F09 - Coach Chatbox
Related Use Case: UC09 - Chat With Coach
Related Sequence Diagram: SD02 - Coach Chatbox and the AI Agent Loop

Classes involved:

- Main: GUI is the Coach Chatbox tab. Sends the user's question and shows the answer with its sources.
- FitnessAppFacade: coordinates the request.
- FitnessAgent: runs the agent loop and keeps a memory list about the user.
- AIModel: the interface the agent uses so it does not matter which AI model answers.
- ClaudeAdapter: connects the agent to Claude Sonnet through the Anthropic Java SDK.
- LlamaAdapter: connects the agent to the local Llama model through Ollama, used as the backup.
- ExerciseLibrary: A Singleton used by the AI agents with a library of exercises.
- WorkoutHistory: the history tool the AI uses to look at past workouts and check-ins.
- DataStore: saves the agent's memory.

Important methods:

- FitnessAppFacade.askCoach()
- FitnessAgent.answerQuestion()
- FitnessAgent.runAgentLoop()
- ClaudeAdapter.isAvailable()
- ClaudeAdapter.send(), or LlamaAdapter.send() as the backup
- FitnessAgent.useTool()
- ExerciseLibrary.search()
- WorkoutHistory.getRecentWorkouts()
- DataStore.saveMemory()

Execution:

When the user sends a question about their progress in the chatbox, Main will call FitnessAppFacade.askCoach() which passes it to FitnessAgent.answerQuestion(). The chatbot will then run FitnessAgent.runAgentLoop() which repeats until the AI gives a final answer about the user's fitness related question. Each time it checks ClaudeAdapter.isAvailable() and sends the prompt with ClaudeAdapter.send(), or with LlamaAdapter.send() if Claude is offline. If the AI asks for a tool then FitnessAgent.useTool() either searches for exercises with ExerciseLibrary.search() or looks at past workouts with WorkoutHistory.getRecentWorkouts() in order to draw from the user's data to be able to give a proper response to their question. Once there is an answer, the agent's memory is saved with DataStore.saveMemory() to revisit whenever and Main shows the answer with its sources to the user.

F10 - Nutrition and Meals

Related Use Case: UC10 - Get Nutrition and Meal Ideas

Related Sequence Diagram: SD08 - Nutrition and Meal Ideas

Classes involved:

- Main: GUI is the Nutrition tab. Shows the targets and the meal ideas with the option to Accept or Regenerate them.
- FitnessAppFacade: coordinates the request.
- UserProfile: calculates the calorie and macro targets.
- FitnessAgent: gets the meal ideas and runs the allergy check on them.
- AIModel: writes the meal ideas.
- DataStore: saves the agent's memory.

Important methods:

- FitnessAppFacade.calculateTargets()
- UserProfile.calculateTargets()
- FitnessAppFacade.getMealIdeas()
- FitnessAgent.createMealIdeas()
- AIModel.send()
- DataStore.saveMemory()

Execution:
When the user calculates their nutritional targets, Main will call FitnessAppFacade.calculateTargets() which uses the formula in UserProfile.calculateTargets() and data from the user profile. When the user asks for meal ideas, FitnessAppFacade.getMealIdeas() calls FitnessAgent.createMealIdeas() which asks the AI for ideas with AIModel.send(). The AI checks its ideas against the nutritional targets with its calculator tool and only gives back the meals that provide enough nutrition and also pass the user's food restriction list. The agent's memory is then saved with DataStore.saveMemory() so the user's food preferences are remembered. The user can keep regenerating meals until they choose to accept one.

F11 - Weekly Progress Report
Related Use Case: UC11 - View Weekly Progress Report
Related Sequence Diagram: SD09 - Weekly Progress Report
Classes involved:

- Main: GUI is the Report tab. Shows the report with the option to Apply or Deny each suggestion and a Save Report button.
- FitnessAppFacade: coordinates the request and sends applied suggestions to revamp the workout plan.
- FitnessAgent: collects the week's data and gets the report from the AI.
- AIModel: writes the review and the suggestions.
- WorkoutHistory: the history tool the AI uses to look at past workouts and check-ins.
- DataStore: saves the report.

Important methods:

- FitnessAppFacade.generateWeeklyReport()
- FitnessAgent.writeWeeklyReport()
- AIModel.send()
- WorkoutHistory.getRecentWorkouts()
- WorkoutHistory.getRecentCheckIns()
- FitnessAppFacade.requestChange()
- FitnessAppFacade.saveReport()
- DataStore.saveReport()

Execution:
When the user chooses to generate the weekly report or when it is automatically called, Main will call FitnessAppFacade.generateWeeklyReport() which calls FitnessAgent.writeWeeklyReport(). The system then asks the AI with AIModel.send() and the AI checks last week's workouts and check-ins from WorkoutHistory.getRecentWorkouts() and WorkoutHistory.getRecentCheckIns(). The AI then writes a review of the week and suggestions to include for the next one. If the user chooses to apply a suggestion, it goes through FitnessAppFacade.requestChange() so next week's plan is changed. If the user wants to keep the report then it is saved with FitnessAppFacade.saveReport() and DataStore.saveReport() for the user to see whenever they wish.

F12 - Import Workout History
Related Use Case: UC12 - Import Workout History
Related Sequence Diagram: SD10 - Import Workout History
Classes involved:

- Main: GUI is the Import tab. Shows the preview and the exercises that could not be matched with the option to Import.
- FitnessAppFacade: coordinates the request.
- DataStore: reads the CSV file and saves the workout history.
- ExerciseLibrary: A library of exercises used to match each exercise name from the file.
- WorkoutHistory: holds all of the user's workouts and tells the observers when a new one is added.
- WorkoutLog: the record of one workout with its logged sets.
- WorkoutObserver: the interface for the classes that get told when a new workout is added.
- WeightProgression: works out the next weights when a new workout is added.

Important methods:

- FitnessAppFacade.importWorkoutHistory()
- DataStore.readCsv()
- ExerciseLibrary.findExercise()
- FitnessAppFacade.confirmImport()
- WorkoutHistory.addWorkout()
- WorkoutHistory.notifyObservers()
- DataStore.saveHistory()

Execution:
When the user chooses to import past workout history from another app using a CSV file, Main will call FitnessAppFacade.importWorkoutHistory(). If the file type is not supported then the user is declined further use. Otherwise, the file is read with DataStore.readCsv() and each exercise name is matched with its equivalent in the system’s exercise library; ExerciseLibrary.findExercise(). Any names that could not be matched are shown to the user so they can match them manually. When the user presses Import, FitnessAppFacade.confirmImport() adds each workout with WorkoutHistory.addWorkout() which calls WorkoutHistory.notifyObservers() so the dashboard and the next weights are updated. Finally, the workout history is saved with DataStore.saveHistory().